

RAG-KnowledgeNexus

企业智能文档问答助手

需求与技术文档

基于 RAG 架构的企业级知识库问答系统

技术栈: Spring Boot 2.7 + Vue 3 + Qdrant + MySQL 8 + MinIO + DashScope

文档版本: v3.0 | 最后更新: 2026-06-11

目 录

RAG-KnowledgeNexus — 企业智能文档问答助手

- 一、项目概述
- 二、技术架构
- 三、功能模块详述
- 四、数据库设计
- 五、接口清单
- 六、部署说明
- 七、面试讲解要点
- 八、项目目录结构
- 九、Qdrant REST API 集成细节
- 十、前端架构概览
- 十一、数据一致性保障
- 十二、安全设计
- 十三、版本与更新

RAG-KnowledgeNexus — 企业智能文档问答助手

基于 RAG (Retrieval-Augmented Generation) 架构的企业级知识库问答系统 技术栈: Spring Boot 2.7 + Vue 3 + Qdrant + MySQL 8 + MinIO + 阿里云 DashScope

一、项目概述

1.1 项目背景

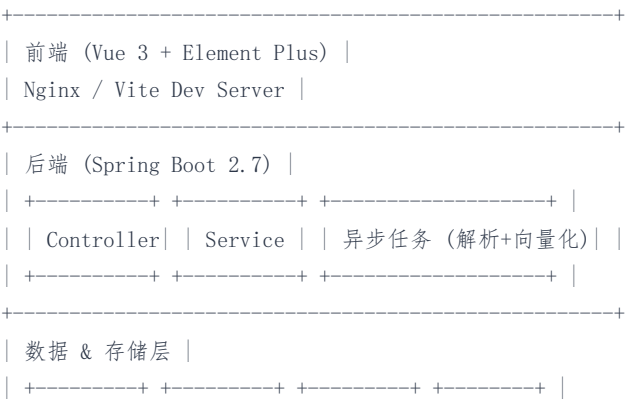
企业内部积累了大量 PDF、Word、TXT 格式的制度和业务文档，员工检索效率低、难以快速获取精准答案。RAG-KnowledgeNexus 将文档向量化存入专业的向量数据库 Qdrant，用户提问时通过向量相似度检索召回相关片段，交由大语言模型（阿里云 Qwen）生成自然语言回答。

1.2 核心能力

模块	功能
用户与权限	管理员 / 知识库管理员 / 普通员工三级角色，部门树形结构，JWT + 接口级鉴权
知识库管理	创建知识库，支持私有 / 部门 / 全员三种可见范围
文档管理	PDF / DOCX / TXT 上传到 MinIO，异步解析 + 语义分块 + 向量化
智能问答	向量检索 + LLM 生成，引用溯源，对话历史管理
工单系统	AI 无法回答时自动流转人工处理
数据统计	日维度指标、热门问题排行、满意度统计

二、技术架构

2.1 总体架构





2.2 技术选型 —— 为什么选 Qdrant

考量维度	选择	理由
部署复杂度	Qdrant	单二进制文件，qdrant.exe 直接启动，无需 Docker / K8s
性能	Qdrant	Rust 编写，HNSW 索引，单机可支撑百万级向量
Java 集成	Qdrant REST API	项目已使用 OkHttp，直接 HTTP 调用，零额外依赖
过滤能力	Qdrant Payload	原生支持标量过滤，可按 kbId、docId 做权限隔离
规模匹配	匹配个人项目	Milvus 适合分布式大规模场景，当前项目 Qdrant 更合适
简历亮点	专业向量数据库	比 Lucene 更专业，比 Milvus 更务实，面试叙事干净

三、功能模块详述

3.1 用户与权限模块

- 登录：账号密码登录，JWT 生成，Token 加入 Redis 黑名单实现登出
- 注册：管理员创建账号（非自由注册），初始密码邮件通知
- 角色体系：
 - 超级管理员（0）：全部权限
 - 知识库管理员（1）：管理指定 KB
 - 普通员工（2）：仅查询
- 部门管理：树形结构，知识库可按部门设置可见范围
- 接口鉴权：@PreAuthorize 注解 + 自定义权限表达式，方法级控制
- 个人中心：密码修改、头像上传（MinIO）

3.2 知识库与文档管理模块

- 知识库 CRUD：名称、描述、可见范围（私有 / 部门 / 全员）
- 文档上传：支持 PDF / DOCX / TXT，文件限制 50MB，同名文件版本覆盖
- 文档列表：分页、状态过滤（解析中 / 完成 / 失败）、关键词搜索
- 文档预览：MinIO 生成预签名 URL，前端直链预览 PDF
- 文档删除：软删除 + 异步清理 Qdrant 向量和 MinIO 文件
- 解析状态：前端轮询获取解析进度

文件处理管道

文件上传 -> 类型判断 -> 文本提取 -> 文本清洗 -> 语义分块 -> 向量嵌入 -> Qdrant 入库

文档解析：

- PDF：Apache PDFBox 3.0.2
- DOCX：Apache POI 5.2.5（支持段落 + 表格提取）
- TXT：UTF-8 直接读取

语义分块策略：

- 按中文标点（。！？\n）切割为自然段落
- 每个 chunk 约 500 字符
- 相邻 chunk 保留 50 字符重叠（overlap），保证语义连续性
- 分块结果写入 MySQL doc_chunk 表，embed_status=0 表示待向量化

3.3 向量化与检索模块

向量化完整链路

第 1 步：文件上传

```
DocumentController.uploadDocument()  
+- 文件 -> MinIO 对象存储（获取 minioPath）  
+- 记录 -> MySQL doc_document 表（parseStatus=0 待处理）  
+- 异步调用 -> documentParseService.parseDocumentAsync(document)
```

第 2 步：文档解析（异步线程池）

```
DocumentParseServiceImpl.parseDocumentAsync() -> parseDocument()
```

通过 @Async("documentParseExecutor") 异步执行

从 MinIO 读取文件流：

```
+- PDF -> PDFBox (Loader.loadPDF)  
+- DOCX -> Apache POI (XWPFDocument) 支持段落 + 表格  
+- TXT -> UTF-8 直接读取  
提取纯文本 content
```

第 3 步：语义分块

```
DocumentParseServiceImpl.semanticChunking()
```

分块策略：

按中文标点（。！？\n）切割成自然段落
遍历段落，累积到当前块
当 currentChunk > 500 字符 -> 生成一个 chunk
保留尾部 50 字符作为 overlap（上下文重叠）
继续下一个块...

每个 chunk:
+- embedStatus = 0 （待向量化）
+- chunkIndex （块序号）
+- content / contentLength
+- docId / kbId
+- qdrantId = null （待填充）
批量插入 MySQL doc_chunk 表
文档状态 -> parseStatus = 2 (SUCCESS)

第 4 步：向量嵌入与 Qdrant 存储
VectorizationServiceImpl.vectorizeChunksByDocId() [异步线程池]
+- chunkMapper.selectByDocId(docId)
+- 过滤 embed_status=0 的分片
+- 分批（25条/批）处理：
| +- EmbeddingServiceImpl.embedBatch(texts) -> HTTP -> 阿里云 DashScope API
| +- QdrantIndexService.upsertVectors(chunkId, vector, payload)
| +- 收集成功/失败
+- chunkMapper.batchUpdateEmbedStatus()
+- 成功的 -> embed_status=1, qdrant_id=xxx
+- 失败的 -> embed_status=2

向量检索 —— Qdrant 替代 Lucene 的改进

原方案（Lucene）的问题：

- 本地文件索引，难以扩展
- 按 kbId 分目录管理，多知识库时维护成本高
- 不支持标量过滤（payload），需要自己在代码里做二次过滤

新方案（Qdrant）的优势：

- 单 Collection 统一管理所有向量，通过 payload 字段（kbId、docId）做过滤
- REST API 调用，与项目现有 OkHttp 风格一致
- Dashboard 可视化查看索引状态
- 单机部署，与 Lucene 运维成本相当

```
// QdrantIndexService 接口（替代 LuceneIndexService）
public interface QdrantIndexService {

    /** 批量写入向量（带 payload: chunkId, kbId, docId） */
    Map<Long, String> upsertVectors(Long kbId, Map<Long, float[]> chunkVectorMap,
    Map<Long, String> chunkPayloadMap);

    /** 向量检索（支持 kbId 过滤） */
}
```

```

List<SearchResult> search(Long kbId, float[] queryVector, int topK);

/** 按知识库删除 */
void deleteByKbId(Long kbId);

class SearchResult {
private final Long chunkId;
private final float score;
}
}

```

3.4 智能问答模块（QA）

检索增强生成（RAG）问答流程

```

POST /api/v1/qa/ask
|
QaController.ask()
|
QaServiceImpl.ask():

```

1. 检查用户可访问的知识库 -> kbIds = [1, 2]
2. EmbeddingService.embed(question) -> float[1024]
| HTTP
阿里云 DashScope text-embedding-v3
3. QdrantIndexService.search(kbId, queryVector, topK=5)
| HTTP
Qdrant ANN 向量检索 -> top-K chunkId 列表
4. 根据 chunkId 从 MySQL 反查完整文本内容
构建上下文 Context = "【文档片段1】...\n【文档片段2】..."

5. 构建 Prompt:

System: "你是一个专业的知识库问答助手。请根据提供的文档内容回答用户的问题。"

要求: 1. 只基于提供的文档内容回答, 不要编造信息。

2. 如果文档内容不足以回答问题, 请明确说明。

3. 回答要简洁、准确、有条理。

4. 如果文档内容包含多个相关点, 请分点说明。"

User: "参考资料: \n" + context + "\n用户问题: " + question

6. ChatService.chat(systemPrompt, userMessage)

| HTTP

阿里云 DashScope qwen-plus -> 生成答案

7. 保存 qa_message (问题 + 答案 + 引用来源 JSON)

8. 返回 { answer, sources, messageId }

回答溯源

每条回答都记录引用的文档片段：

```
{
  "sources": [
    {
      "chunkId": 123,
      "docId": 45,
      "docTitle": "员工考勤制度.pdf",
      "content": "员工每日工作时间为...",
      "score": 0.92
    }
  ]
}
```

3.5 工单系统

AI 无法回答时自动流转人工处理：

状态流转：0(待处理) -> 1(处理中) -> 2(已解决) -> 3(关闭)

- AI 答案质量低分时自动创建工单
- 支持从 QA 消息直接转工单
- 工单指派给指定管理员处理

四、数据库设计

4.1 表结构总览（12 张表，5 层架构）

第一层：用户与权限体系（3 张表）

表名	说明	关键字段
sys_dept	部门表（树形结构）	parent_id（自关联上级）
sys_user	用户表	role: 0=超管, 1=KB管理员, 2=普通员工
sys_operation_log	操作日志	审计追踪

第二层：知识库与文档体系（3 张表）—— RAG 数据基础

表名	说明	关键字段
doc_knowledge_base	知识库表	visibility: 0=私有, 1=部门, 2=全员
doc_document	文档表	parse_status: 0=待处理, 1=解析中, 2=完成, 3=失败
doc_chunk	分片表【核心】	embed_status, qdrant_id, content (FULLTEXT)

第三层：问答与对话体系（3 张表）

表名	说明	关键字段
qa_conversation	对话会话表	用户多次对话
qa_message	问答消息表【核心】	Q/A/评分/溯源(source_chunks JSON)
qa_feedback	答案反馈表	点赞/踩, 评估 RAG 质量

第四层：工单管理体系（1 张表）

表名	说明	关键字段
ticket_order	咨询工单表	status: 0=待处理, 1=处理中, 2=已解决, 3=关闭

第五层：数据统计体系（2 张表）

表名	说明	关键字段
stat_daily	每日统计汇总	positive_rate（好评率）
stat_hot_question	热门问题统计	question_hash（MD5 去重）

4.2 核心设计思路

为什么在 `doc\_chunk` 冗余 `kb\_id`？

```
-- 差的设计（需要 3 表 JOIN）
SELECT * FROM doc_chunk dc
JOIN doc_document dd ON dc.doc_id = dd.id
JOIN doc_knowledge_base dkb ON dd.kb_id = dkb.id
WHERE dkb.id IN (用户可访问的 KB 列表)
LIMIT 10;
```

```
-- 好的设计（单表查询，冗余 kb_id）
SELECT * FROM doc_chunk
WHERE kb_id IN (用户可访问的 KB 列表)
LIMIT 10;
```

冗余 kb_id 是安全的 —— 分片一旦创建，所属知识库关系就固定了，不会改变。

为什么 `qa\_message.source\_chunks` 用 JSON？

JSON 方案（推荐）：

```
SELECT * FROM qa_message WHERE id = 123;
-- 一次查询获得完整的 Q、A、引用信息
```

JSON 格式灵活，可存储 [{chunk_id, doc_id, doc_title, score}], 一次查询完成。不需要额外关联表。

权限隔离如何设计？

知识库有三种可见范围，在数据库层完成权限过滤（WHERE 条件），而非应用层事后过滤：

可见范围	权限条件	SQL WHERE 子句
私有(0)	仅创建人可见	kb.owner_id = ?

可见范围	权限条件	SQL WHERE 子句
部门(1)	指定部门的员工	kb.dept_id = user.dept_id
全员(2)	所有员工可见	无额外过滤

这样做的好处：

- 避免无效的数据库查询（检索已按权限过滤）
- 防止权限泄漏（应用层过滤容易遗漏）
- 性能最优（数据库索引加速）

4.3 外键约束一览

父表	子表	删除策略	说明
sys_user	sys_user.dept_id	SET NULL	员工所属部门
sys_user	doc_knowledge_base.owner_id	CASCADE	知识库创建者
sys_dept	doc_knowledge_base.dept_id	SET NULL	部门可见性
doc_knowledge_base	doc_document.kb_id	CASCADE	文档属于 KB
doc_document	doc_chunk.doc_id	CASCADE	分片属于文档
doc_knowledge_base	doc_chunk.kb_id	CASCADE	分片属于 KB
sys_user	qa_conversation.user_id	CASCADE	会话用户
doc_knowledge_base	qa_conversation.kb_id	CASCADE	会话 KB
qa_conversation	qa_message.conversation_id	CASCADE	消息会话
qa_message	qa_feedback.message_id	CASCADE	反馈消息
qa_message	ticket_order.message_id	SET NULL	工单来源 QA
sys_user	ticket_order.assignee_id	SET NULL	工单处理人

删除策略说明：

- CASCADE：级联删除（删除父记录时自动删除子记录）
- SET NULL：置空删除（删除父记录时将子记录的 FK 设为 NULL）
- RESTRICT：拒绝删除（存在子记录时拒绝删除父记录）

4.4 关键索引

```
-- 主键索引（自动）
PRIMARY KEY (id)

-- 单列索引
KEY idx_user_id (user_id)
KEY idx_kb_id (kb_id)
```

```

KEY idx_doc_id (doc_id)
KEY idx_create_time (create_time)
KEY idx_status (status)

-- 组合索引（高频联合查询）
KEY idx_user_kb_time (user_id, kb_id, create_time)
KEY idx_kb_status_id (kb_id, embed_status, id)

-- 全文索引（BM25 关键词搜索）
FULLTEXT KEY ft_content (content) WITH PARSER ngram

```

4.5 Qdrant Collection 映射

MySQL doc_chunk 表与 Qdrant 的映射关系：

MySQL doc_chunk	Qdrant Collection	说明
id (chunkId)	id (Point ID)	分片唯一标识
content	payload.text	分片文本（存入 payload 便于调试）
embedding	vector (1024-dim)	向量数据
doc_id	payload.doc_id	文档 ID，用于过滤
kb_id	payload.kb_id	知识库 ID，权限过滤
chunk_index	payload.chunk_index	分片序号

检索流程：

1. 问题向量化 -> 在 Qdrant 中 ANN 搜索（带 kb_id 过滤条件）
2. 得到匹配的 Point ID (= chunkId) 列表
3. 通过 chunkId 反查 MySQL doc_chunk 获取完整文本
4. 拼接 Prompt 发送给 LLM

五、接口清单

5.1 用户模块

方法	路径	说明
POST	/api/v1/user/login	用户登录
POST	/api/v1/user/register	管理员创建用户
GET	/api/v1/user/info	获取当前用户信息
PUT	/api/v1/user/update	修改个人信息
GET	/api/v1/user/list	用户列表（分页）
GET	/api/v1/dept/tree	部门树

5.2 知识库模块

方法	路径	说明
POST	/api/v1/knowledge	创建知识库
GET	/api/v1/knowledge/list	知识库列表（权限过滤）
PUT	/api/v1/knowledge/{id}	编辑知识库
DELETE	/api/v1/knowledge/{id}	删除知识库（软删除）
POST	/api/v1/knowledge/{id}/share	分享知识库

5.3 文档模块

方法	路径	说明
POST	/api/v1/document/upload	上传文档（表单，file + kbId）
GET	/api/v1/document/list	文档列表（分页，按 KB 过滤）
GET	/api/v1/document/{id}	文档详情
DELETE	/api/v1/document/{id}	删除文档（软删除）
GET	/api/v1/document/{id}/chunks	查看文档分片

5.4 问答模块

方法	路径	说明
POST	/api/v1/qa/conversation	创建对话
POST	/api/v1/qa/ask	提问（RAG 核心接口）
GET	/api/v1/qa/conversation/list	对话列表
GET	/api/v1/qa/conversation/{id}	对话详情（含消息历史）
POST	/api/v1/qa/feedback	答案反馈（赞/踩）

六、部署说明

6.1 环境依赖

组件	版本	端口	用途
JDK	17+	-	Java 运行时
MySQL	8.0	3306	业务数据库
Qdrant	1.x	6333 (HTTP) / 6334 (gRPC)	向量数据库
MinIO	latest	9000	对象存储
Redis	7.x	6379	Token 黑名单

6.2 启动步骤

```
# 1. 启动依赖服务
# MySQL、Redis、MinIO、Qdrant

# 2. 初始化数据库
mysql -u root -p rag_knowledge_base < rag_knowledge_base_ddl.sql

# 3. 配置 application-dev.yml
# 修改数据库连接、MinIO 地址、Embedding API Key 等

# 4. 后端启动
cd backend/rag-knowledgenexus/rag-doc-server
mvn spring-boot:run
# 服务运行在 http://localhost:8085

# 5. 前端启动
cd frontend
npm install
npm run dev
# 开发服务器运行在 http://localhost:5173
```

6.3 配置文件说明

核心配置项（application-dev.yml）：

```
rag:
  minio:
    endpoint: http://192.168.150.101:9000
    access-key: minioadmin
    secret-key: minioadmin
    bucket:
    document: rag-documents
    avatar: user-avatar

  embedding:
    provider: aliyun
    model: text-embedding-v3
    api-key: sk-xxx
    api-url: https://dashscope.aliyuncs.com/compatible-mode/v1/embeddings

  chat:
    provider: aliyun
    model: qwen-plus
    api-key: sk-xxx
    api-url: https://dashscope.aliyuncs.com/compatible-mode/v1/chat/completions

  qdrant:
    host: localhost
    port: 6334
    collection: rag_knowledge_chunks
```

vector-size: 1024

七、面试讲解要点

7.1 为什么选 Qdrant 而不是 Lucene 或 Milvus?

Lucene 是全文搜索引擎，向量检索是 Lucene 9.x 才后加的功能。它本质上是为倒排索引设计的，向量检索用本地文件管理，缺乏 Collection 概念和标量过滤，也不支持可视化管理。

Milvus 是专业的分布式向量数据库，但它适合百万级以上向量、集群部署的场景。个人项目规模下引入 Milvus 显得过度设计。

Qdrant 是中间最优解：

- 真正的向量数据库（Collection、Payload 过滤、ANN 检索）
- 单二进制部署，运维成本与 Lucene 相当
- Rust 编写，单机性能出色
- 简历上写“Qdrant”比写“Lucene”专业得多，同时能让面试官感受到你做技术选型时有务实判断

7.2 混合检索

不要只用向量检索。加上 BM25 关键词检索（MySQL FULLTEXT），两路结果用 RRF 算法融合排序：

```
向量检索 Top-K + BM25 关键词检索 Top-K
|
RRF 融合排序: score = 1/(60+rank_vector) + 1/(60+rank_bm25)
|
最终结果
```

为什么好？纯向量检索对精确词汇（如产品型号“XR-7800-Pro”）效果差，混合检索是工业界标准做法。

7.3 权限隔离检索

不是在检索完后过滤结果，而是在检索时就带上权限条件：

```
SELECT * FROM doc_chunk
WHERE kb_id IN (用户有权限访问的 KB 列表)
AND is_deleted = 0
LIMIT 8
```

这样既安全又高效，是企业级系统的体现。

7.4 RAG 效果评估

通过 qa_feedback 和 stat_hot_question 统计用户反馈：

```
SELECT question, ask_count, positive_rate
FROM stat_hot_question
```

```
WHERE positive_rate < 30
ORDER BY ask_count DESC;
```

系统不是黑盒，管理员可以看到哪些问题效果差，并针对性优化文档。

八、项目目录结构

```
RAG-KnowledgeNexus/
├── backend/
│   ├── rag-knowledgenexus/
│   ├── rag-doc-common/ # 公共模块（工具类、常量）
│   ├── rag-doc-model/ # 数据模型（实体、DTO、VO）
│   │   ├── src/main/java/com/xxr/
│   │   ├── document/pojo/ # 文档相关实体（DocChunk, DocDocument）
│   │   ├── kb/pojo/ # 知识库实体（KnowledgeBase）
│   │   ├── qa/pojo/ # 问答实体（QaMessage, QaConversation）
│   │   ├── user/pojo/ # 用户实体（SysUser, SysDept）
│   │   └── ...
│   ├── rag-doc-server/ # 主服务模块
│   ├── src/main/java/com/xxr/
│   ├── config/ # 配置类（Minio, Security, WebMvc, Async）
│   ├── controller/ # REST 控制器
│   ├── interceptor/ # JWT 拦截器
│   ├── mapper/ # MyBatis-Plus 映射
│   ├── security/ # 安全配置
│   ├── service/ # 业务服务
│   ├── impl/ # 服务实现
│   ├── QdrantIndexService.java # Qdrant 向量检索
│   ├── VectorizationService.java # 向量化编排
│   ├── EmbeddingService.java # Embedding API 调用
│   ├── ChatService.java # LLM 聊天
│   ├── DocumentService.java # 文档管理
│   ├── KnowledgeService.java # 知识库管理
│   └── ...
├── frontend/ # 前端（Vue 3 + Element Plus + TypeScript）
│   ├── src/
│   │   ├── api/ # API 接口封装（axios）
│   │   ├── components/ # 公共组件
│   │   ├── views/ # 页面视图
│   │   │   ├── knowledge/ # 知识库管理页面
│   │   │   ├── document/ # 文档管理页面
│   │   │   ├── qa/ # 问答页面
│   │   │   ├── ticket/ # 工单管理页面
│   │   │   └── user/ # 用户管理页面
│   │   ├── router/ # 路由配置
│   │   ├── stores/ # Pinia 状态管理
│   │   └── utils/ # 工具函数
│   └── ...
└── files/ # 文档目录
```

- | |—— 技术文档.md # 本技术文档
- | |—— README-数据库设计使用指南.md # 数据库设计指南
- | |—— rag_knowledge_base_ddl.sql # 数据库初始化 SQL
- | |—— RAG-KnowledgeNexus-* # 生成的 PDF 文档
- |—— logs/ # 运行时日志

九、Qdrant REST API 集成细节

系统通过 OkHttpClient 直接调用 Qdrant REST API，无需额外依赖。核心调用示例：

```
// Qdrant REST API 端点
String baseUrl = "http://localhost:6333";
String collection = "rag_knowledge_chunks";

// 1. 创建 Collection
POST /collections/{name}
{
  "vectors": { "size": 1024, "distance": "Cosine" }
}

// 2. Upsert 向量 (带 Payload)
PUT /collections/{name}/points
{
  "points": [{
    "id": 123,
    "vector": [0.1, 0.2, ...],
    "payload": {
      "kb_id": 1,
      "doc_id": 45,
      "chunk_index": 0
    }
  }],
  "wait": true
}

// 3. 带过滤的 ANN 搜索
POST /collections/{name}/points/search
{
  "vector": [0.1, 0.2, ...],
  "limit": 5,
  "filter": {
    "must": [
      { "key": "kb_id", "match": { "value": 1 } }
    ]
  }
}

// 4. 按过滤条件删除
POST /collections/{name}/points/delete
```

```
{
  "filter": {
    "must": [
      { "key": "doc_id", "match": { "value": 45 } }
    ]
  }
}
```

十、前端架构概览

- 框架：Vue 3 + Composition API
- UI 库：Element Plus
- 构建工具：Vite
- 语言：TypeScript
- 状态管理：Pinia
- HTTP 客户端：Axios（带请求/响应拦截器）

页面路由：

路径	页面	说明
/login	登录页	账号密码登录
/	工作台	仪表盘与快捷操作
/knowledge	知识库列表	查看和管理知识库
/knowledge/:id	知识库详情	文档列表与管理
/document/:id	文档详情	文档预览与分片查看
/qa/:kbId	问答页面	对话式问答交互
/ticket	工单管理	工单列表与处理
/user	用户管理	管理员管理用户

十一、数据一致性保障

写入链路（MySQL + Qdrant 双写）

1. 文件上传 → MySQL 写入 doc_document (parse_status=0)
2. 异步解析 → 语义分块 → MySQL 批量插入 doc_chunk (embed_status=0)
3. 异步向量化 → DashScope Embedding API → 获取向量
4. OK → Qdrant upsert → 更新 embed_status=1 + vector_id
5. FAIL → embed_status=2（失败），保留重试机制

删除链路

1. 用户删除文档 → MySQL 软删除 doc_document + doc_chunk (is_deleted=1)
2. 异步清理 → Qdrant delete_points (按 doc_id 过滤)
3. 异步清理 → MinIO 删除文件

十二、安全设计

- 密码加密: BCrypt 哈希存储
- 身份认证: JWT (Access Token + Refresh Token)
- Token 管理: 登录生成 Token, 登出加入 Redis 黑名单
- 接口鉴权: Spring Security + @PreAuthorize 注解, 方法级权限控制
- 数据权限: SQL 层权限过滤 (WHERE kb_id IN ...), 防止越权访问
- 文件安全: MinIO 预签名 URL, 防止直接访问存储桶
- 防注入: MyBatis-Plus 参数绑定, 避免 SQL 注入

十三、版本与更新

- 版本: v2.0 (Qdrant 版)
- 技术文档更新日期: 2025-06-11
- 作者: 个人项目
- 变更记录:
- v2.0: 向量数据库从 Lucene 迁移至 Qdrant, 文档全面更新
- v1.0: 初始版本, 基于 Lucene 向量检索